

Mekanisme expected_delay_hours Saat Ini

Bagaimana field ini didefinisikan, kapan bernilai nol vs tidak nol, bagaimana ia dipakai model XGBoost, dan bagaimana data training-nya dibangun per moda transportasi — dasar sebelum menambahkan estimasi delay berbasis cuaca (checklist proposal No. 13).

Branch: fix/hackathon-readiness

Service: services/ai/route-planner

Tanggal dokumentasi: 22 Agustus 2026

Konteks: riset awal sebelum implementasi fix checklist No. 13

1. Dua Field yang Mirip Tapi Berbeda Peran

Sistem JaGOOD punya dua field terkait "delay" yang sering tertukar — penting dipahami dulu sebelum menambah fitur cuaca:

Field	Peran	Sumber nilai
<code>expected_delay_hours</code>	Delay tambahan khusus permintaan ini — ditambahkan langsung ke durasi perjalanan pengiriman yang sedang diproses.	Parameter input per-request ke <code>enrich_candidate()</code> ; default 0 .
<code>historical_delay_avg_hours</code>	Estimasi delay tipikal untuk profil koridor ini (moda + jarak + eksposur) — fitur risiko, bukan delay pengiriman yang sedang diproses.	Formula deterministik di <code>historical_baseline.py</code> , kini juga di-blend dengan data outcome nyata (lihat dokumentasi Corridor Baseline Learning).

KENAPA INI PENTING UNTUK FITUR CUACA YANG DIRENCANAKAN

Rencana delay berbasis cuaca (checklist No. 13) akan mengubah `expected_delay_hours` , bukan `historical_delay_avg_hours` . Field ini yang langsung mempengaruhi `estimated_arrival` , simulasi suhu kargo (`effective_consumed_hours`), dan merupakan fitur numerik yang masuk ke model XGBoost — jadi dampaknya lebih langsung dan lebih berisiko menyebabkan train/serve skew kalau distribusi nilainya berubah drastis di serving time.

2. Skema & Definisi

`app/schemas/route_schema.py` — `RouteCandidate.expected_delay_hours` :

```
expected_delay_hours: float = Field(
    default=0,
    description="Additional expected in-transit delay beyond the route's normal travel duration.",
)
```

`app/ml/feature_pipeline.py` — termasuk dalam `NUMERIC_FEATURES` yang diteruskan langsung ke `XGBClassifier` (tanpa normalisasi, sesuai gaya fitur numerik lain di pipeline ini).

3. Alur Serving: Kapan Bernilai Nol, Kapan Tidak



Endpoint	Nilai expected_delay_hours	Sumber kode
POST /predict-route (normal)	Selalu 0	planning_service.build_route_plan() memanggil evaluate_candidates() tanpa override — memakai default parameter additional_delay_hours: float = 0.
POST /simulate-scenario (baseline)	Selalu 0	Cabang baseline di scenario_service.py juga tidak mengoper delay_hours.
POST /simulate-scenario (simulated)	Sesuai input user	Cabang simulated mengoper additional_delay_hours=request.changes.delay_hours — nilai yang secara eksplisit diminta user lewat form "Delay tambahan (jam)" di Scenario Simulator.

IMPLIKASI PRAKTIS HARI INI

Di luar simulasi skenario yang sengaja di-trigger user, **tidak ada prediksi rute normal yang pernah menunjukkan delay non-nol** — terlepas dari kondisi cuaca/gelombang apa pun yang terdeteksi di rute tersebut. Ini persis temuan checklist No. 13: durasi rute dihitung, tapi penyesuaian keterlambatan berdasarkan lingkungan belum benar-benar terjadi di jalur prediksi utama.

4. Bagaimana Nilainya Dipakai Setelah Dihitung

app/services/enrichment_service.py — begitu expected_delay_hours masuk ke enrich_candidate(), ia dipakai di tiga tempat:

- Durasi total & waktu tiba:** duration = estimated_duration_hours + expected_delay_hours ;
estimated_arrival = departure_time + timedelta(hours=duration) .
- Simulasi suhu kargo (moda pasif):** duration di atas jadi total_duration_hours untuk simulate_cargo_temperature() — memperpanjang delay berarti kargo terpapar suhu ambient lebih lama, langsung mempengaruhi remaining_shelf_life_hours / quality_status (fitur yang baru saja diekspos, lihat dokumentasi Estimated Product Quality).
- Fitur model risiko:** nilai yang sama diteruskan apa adanya sebagai kolom expected_delay_hours ke ranking_service.predict_risk() lewat feature_pipeline.NUMERIC_FEATURES .

5. Bagaimana Model Belajar dari expected_delay_hours Saat Training

training/generate_synthetic_data.py membangkitkan nilai delay per baris training **secara berbeda tergantung moda** — ini titik krusial untuk rencana fitur cuaca:

Moda	Formula delay training	Terkait cuaca?
laut / kombinasi	<code>corridor.base_delay_hours + CONDITION_DELAY_BONUS[wave_category] + (3.0 jika "Badai" di weather_condition) + noise</code> , dibatasi ≥ 0	Ya — delay training sudah bervariasi mengikuti kategori gelombang & badai
darat	<code>corridor.base_delay_hours + noise</code> (noise kecil, $\sigma \approx 1.5$ jam)	Tidak — delay training darat sepenuhnya independen dari cuaca

TEMUAN RISIKO UTAMA UNTUK RENCANA SELANJUTNYA

Karena delay training darat tidak pernah berkorelasi dengan cuaca, model XGBoost **tidak pernah belajar pola** "hujan deras + darat $\rightarrow$ delay tinggi" untuk kombinasi itu. Kalau fitur delay berbasis cuaca diaktifkan di serving time untuk rute darat tanpa menyesuaikan data training, nilai `expected_delay_hours` yang dikirim ke model akan berada di rentang/pola yang belum pernah dilihat model untuk moda darat secara spesifik — berisiko prediksi risiko yang kurang bisa diandalkan untuk kasus itu (train/serve skew). Risiko ini **tidak berlaku sama besarnya untuk laut/kombinasi**, karena training moda itu memang sudah mengandung variasi delay terkait cuaca.

`expected_delay_hours` training-time (disebut `shipment_delay_hours` dalam skrip) juga langsung mempengaruhi label risiko lewat `delay_tolerance_ratio` (dibandingkan terhadap `commodity_delay_tolerance_hours`) — jadi field ini bukan sekadar fitur pasif, tapi salah satu pendorong utama label `risk_level` saat training.

6. Ringkasan untuk Pengambilan Keputusan Berikutnya

- Infrastruktur pendukung sudah ada: geometri rute darat dari ORS (`route_generator.py`) sudah cukup detail untuk sampling cuaca sepanjang jalur, memakai pola yang sama dengan `_sample_geometry()` yang sudah dipakai simulasi suhu kargo pasif.
- Open-Meteo (sudah terintegrasi untuk suhu ambient pasif) bisa diperluas dengan parameter `precipitation / weathercode` di request `hourly` yang sama — tidak perlu integrasi API baru untuk mendapat sinyal hujan di rute darat.
- BMKG (laut) sudah mengirim `weather_condition` bertingkat (Cerah $\rightarrow$ Hujan Badai), tapi belum ada pemetaan eksplisit tingkat keparahan $\rightarrow$ menit keterlambatan — ini yang perlu dibuat sebagai tabel aturan sintetis (rule-based), sesuai rekomendasi checklist No. 13.
- Keputusan terbuka: apakah delay cuaca ini langsung mengubah `expected_delay_hours` (mempengaruhi model, dengan risiko skew darat seperti dijelaskan di atas) atau disurfacekan sebagai field transparan terpisah dulu (lebih aman, sesuai kalimat checklist "baseline delay transparan"), dengan opsi retrain model kapan pun setelah generator data training ikut disesuaikan.